

CyberAudit & Solutions

Documentation technique du MVP

Stage 3 — Réutilisable pour le RNCP 5
Développeur Web et Web Mobile

Référent technique	Tommy JOUHANS	Développeur Front-End
Référent technique	James ROUSSEL	Développeur Back-End

Sommaire

1. Introduction

- 1.1 Contexte
- 1.2 Périmètre du document
- 1.3 Public visé
- 1.4 Stack technique

2. User Stories & Mockups

- 2.1 Personas
- 2.2 User Stories priorisées (MoSCoW)
- 2.3 Description des écrans (11 mockups)

3. Architecture du système

- 3.1 Vue d'ensemble — Architecture en couches
- 3.2 Flux de données — Demande d'audit
- 3.3 Choix d'architecture justifiés
- 3.4 Accessibilité (WCAG 2.1 AA)
- 3.5 Budget de performance (Web Vitals)

4. Composants, classes et BDD

- 4.1 Schéma BDD
- 4.2 Diagramme de classes back-end
- 4.3 Composants front-end (React + Tailwind)
- 4.4 Génération du rapport PDF
- 4.5 Monitoring & observabilité
- 4.6 Migrations BDD & seeding

5. Diagrammes de séquence

- 5.1 Inscription + Connexion JWT
- 5.2 Soumission demande d'audit (async)
- 5.3 Génération du rapport PDF (admin)

6. Documentation des APIs

- 6.1 APIs externes consommées
- 6.2 API interne — Endpoints REST
- 6.3 Conventions transversales

7. Stratégie SCM & QA

- 7.1 Source Code Management (Git)
- 7.2 Stratégie QA
- 7.3 Tests sécurité manuels
- 7.4 Pipeline CI/CD
- 7.5 Conformité RGPD

8. Annexes

- 8.1 Glossaire
- 8.2 Mapping Stage 3 ↔ RNCP 5
- 8.3 Sources & références techniques
- 8.4 Matrice des risques projet
- 8.5 Liens internes au projet

1. Introduction

1.1 Contexte

CyberAudit & Solutions est une plateforme web full-stack qui connecte les PME/TPE françaises sans équipe IT interne avec des experts en cybersécurité. Selon l'ANSSI 2023, 45 % des PME n'ont aucune équipe informatique dédiée et restent exposées à des cyberattaques (ransomware, phishing, fuite de données).

1.2 Périmètre du document

Cette documentation technique couvre la conception détaillée du MVP. Elle décrit :

- Les besoins utilisateurs formalisés en User Stories priorisées
- L'architecture logicielle et le flux de données
- Le modèle de données (PostgreSQL relationnel)
- Les classes back-end, composants front-end, scénarios d'interaction
- Les APIs exposées et consommées
- La stratégie de gestion du code et de qualité

1.3 Public visé

- Équipe de développement (Tommy + James) — référence pendant la phase de codage (W2 → W12)
- Jury Holberton — preuve de la conception détaillée du MVP
- Jury RNCP 5 — dossier de compétences réutilisable pour la certification Développeur Web et Web Mobile

1.4 Stack technique

Couche	Technologie	Justification
Front-End	React 18 + Tailwind CSS	SPA performante, design system rapide
Back-End	Django 5 + DRF	Sécurité native, écosystème mature
BDD	PostgreSQL 15	Relationnel ACID, index performants
Async	Celery + Redis	Tâches longues hors thread HTTP
Auth	SimpleJWT	Stateless, scalable
Hébergement	Vercel + Railway	HTTPS auto, déploiement Git

2. User Stories & Mockups

2.1 Personas

Marie — 45 ans, gérante d'un cabinet comptable à Dijon, aucune notion IT. Cible UX < 3 min.

Karim — 32 ans, expert cybersécurité, gère la file de demandes en parallèle.

2.2 User Stories priorisées (MoSCoW)

Must Have

ID	User Story	Stage
US-01	En tant que visiteur, je veux m'inscrire, afin de créer mon compte.	W3
US-02	En tant qu'utilisateur, je veux me connecter, afin d'accéder à mon espace.	W3
US-03	En tant qu'utilisateur, je veux me déconnecter, afin de protéger mon compte.	W3
US-04	En tant que cliente PME, je veux soumettre une demande en < 3 min.	W4
US-05	En tant que cliente, je veux suivre le statut (4 statuts).	W4
US-06	En tant que cliente, je veux recevoir un email à chaque changement.	W4
US-07	En tant qu'admin, je veux voir toutes les demandes triées.	W5
US-08	En tant qu'admin, je veux changer le statut.	W5
US-09	En tant qu'admin, je veux générer un PDF.	W5
US-10	En tant que cliente, je veux télécharger mon PDF.	W5

Should Have

US-11 modules sensibilisation • US-12 reset MDP • US-13 filtres admin • US-14 notifications UI.

Could Have / Won't Have

US-15 à US-17 (V2). Hors périmètre : app mobile, score IA, marketplace, OAuth2, gamification.

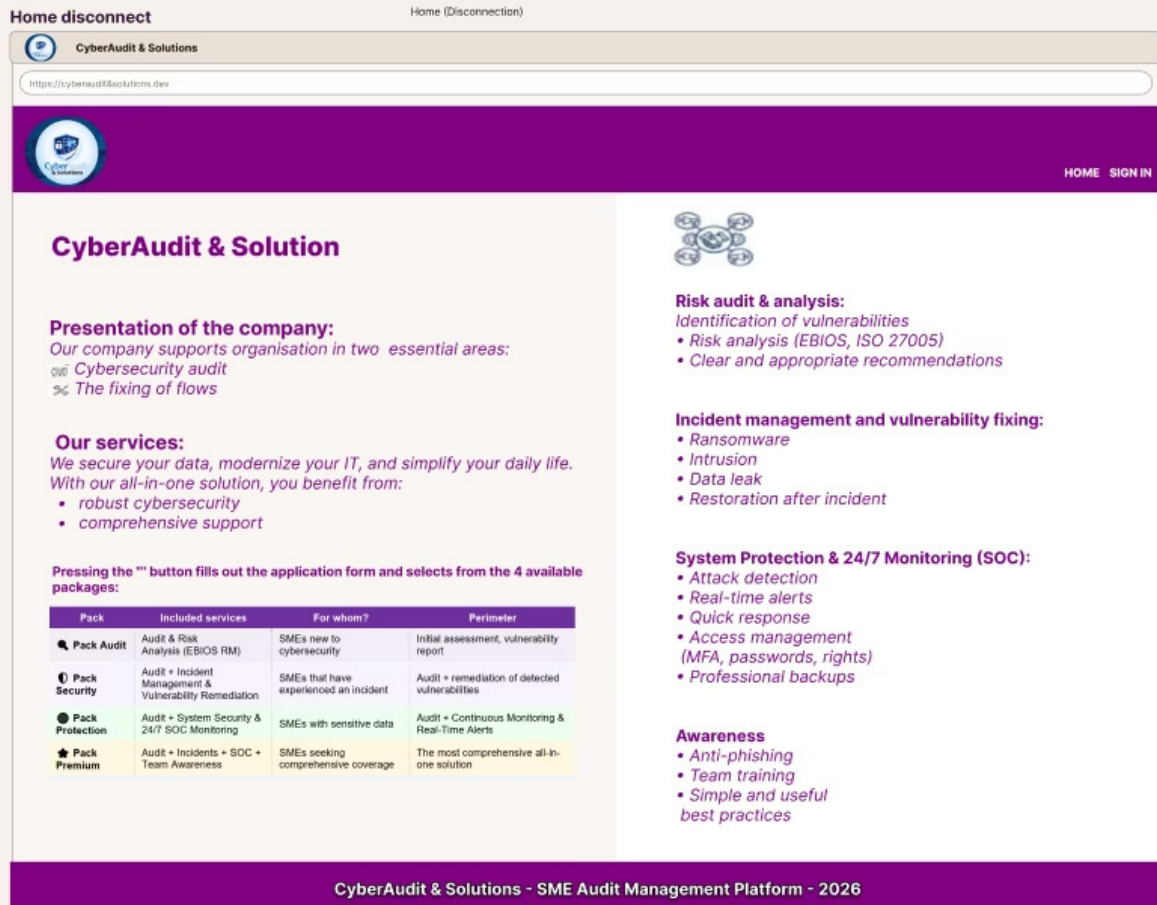
2.3 Description des écrans (11 mockups Figma à produire en W1)

Écran 1 — Landing publique

Hero (problème ANSSI), CTA, bénéfices. La page d'accueil de la plateforme CyberAudit & Solutions présente l'entreprise, les prestations de services et les packs proposés adaptés aux entrepreneurs de moins de 50 salariés. Il suffit d'aller sur l'onglet « SIGN IN » pour que la cliente (Marie) puisse créer un compte et se connecter.

1. Public Access & Authentication

Screens visible before login: public home, login/registration, logged-in home



Mockup 1 — Landing publique

Écran 2 — Inscription

Email + MDP + CGU/RGPD. Après avoir cliqué sur « Create an account », la cliente renseigne le nom de son entreprise, son prénom, son nom, son email et son mot de passe (haché). Règles de remplissage :

- Chaque champ contient au maximum 50 caractères
- Si une case est manquante : erreur « veuillez saisir l'information sur le formulaire »
- Email format `nom@domaine` obligatoire
- Mot de passe : une minuscule, une majuscule, un chiffre, un caractère spécial, minimum 10 caractères

Create an account

Log in

CyberAudit & Solutions

https://cyberaudit&solutions.dev

CyberAudit & Solutions

HOME SIGN IN

Create an account

Fill in the information below to get started

Company Name

Enter company name

First Name

Enter first name

Last Name

Enter last name

Email

Enter email address

Password

Create a password

Create an account

Already have an account?

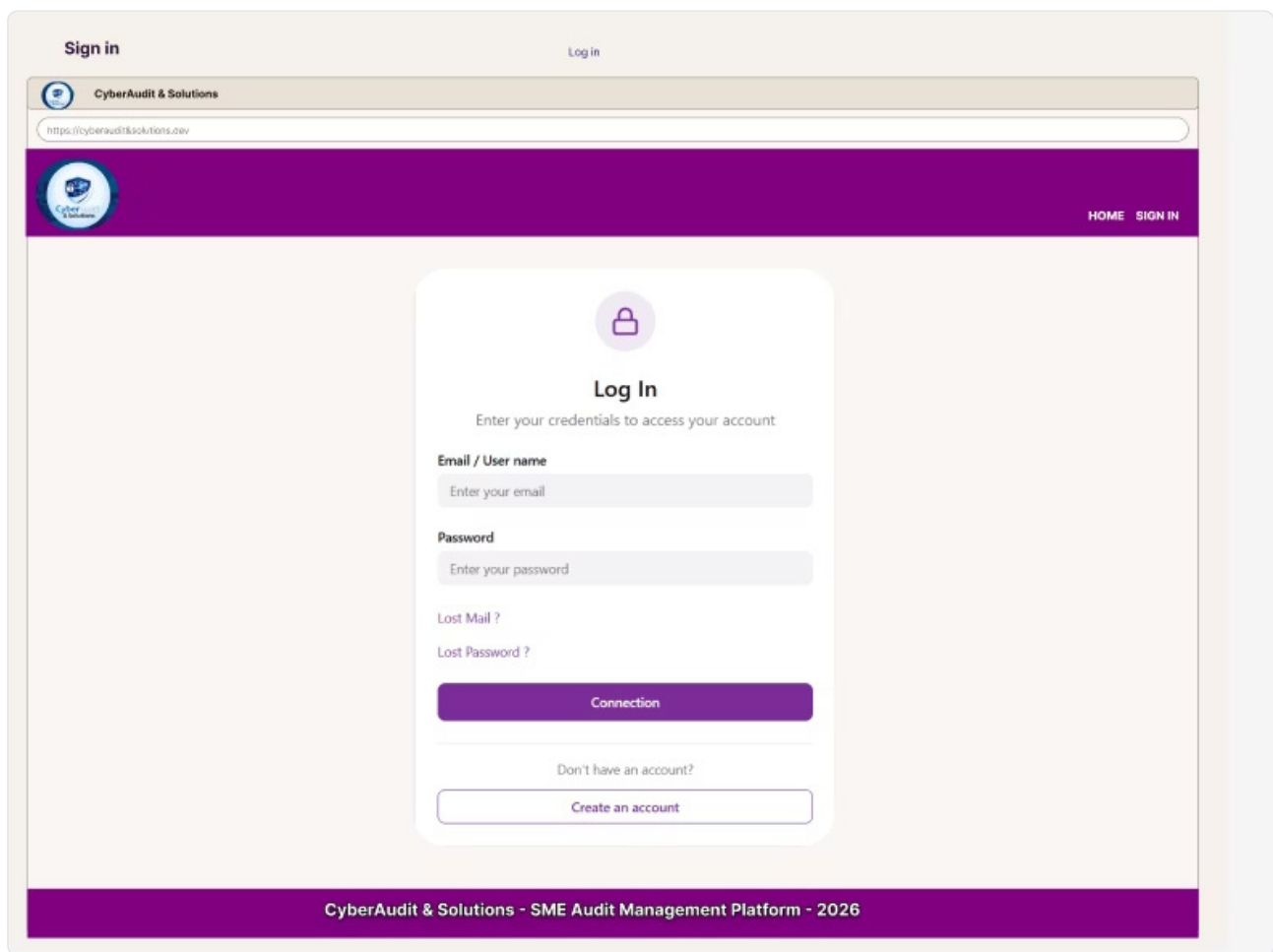
Sign In

CyberAudit & Solutions - SME Audit Management Platform - 2026

Mockup 2 — Inscription

Écran 3 — Connexion

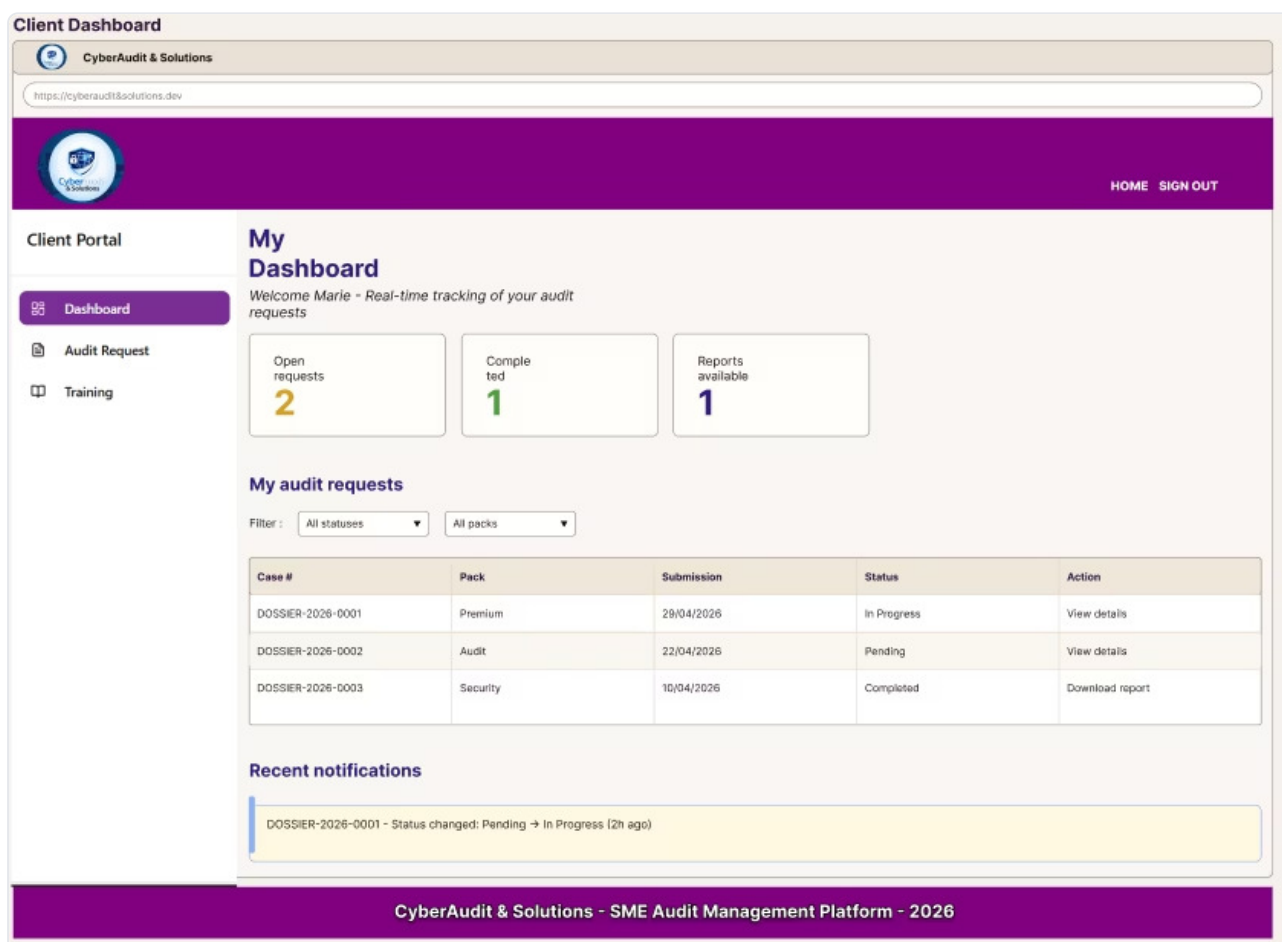
Email + MDP. La cliente peut se connecter avec son compte ; sinon elle clique sur « Create an account ». Liens « Lost Mail » et « Lost Password » prévus (à développer ultérieurement pour la présentation RNCP 5).



Mockup 3 — Connexion

Écran 4 — Dashboard Client

Sidebar, liste de demandes, CTA flottant. Lorsque la cliente est connectée, elle consulte son tableau de bord pour suivre l'avancement de sa demande d'audit (En attente / En cours / Terminé). Elle peut cliquer sur « View details » et télécharger le rapport via « Download report ».



Mockup 4 — Dashboard Client

Écran 5 — Formulaire d'audit

Stepper 3 étapes, cible < 3 min. La cliente renseigne nom d'utilisateur, nom d'entreprise, choix du pack (Audit / Security / Protection / Premium) avec affichage des « Services inclus », et un message optionnel. Bouton « Send the Audit Request » pour envoyer.

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

Client Portal

- Dashboard
- Audit Request
- Training

Audit request form

Username :

Company Name :

Select pack :

- ☒ Pack Audit
- ☐ Pack Security
- ☐ Pack Protection
- ☐ Pack Premium

Included Services:

Print in the panel of included services of the selected package such as:
If you select "Audit & Risk"
Display for example:
Analysis (EBIOS RM) | SMEs new to cybersecurity | Initial assessment, vulnerability report | 5 business day | Price: 1000 €

Message :

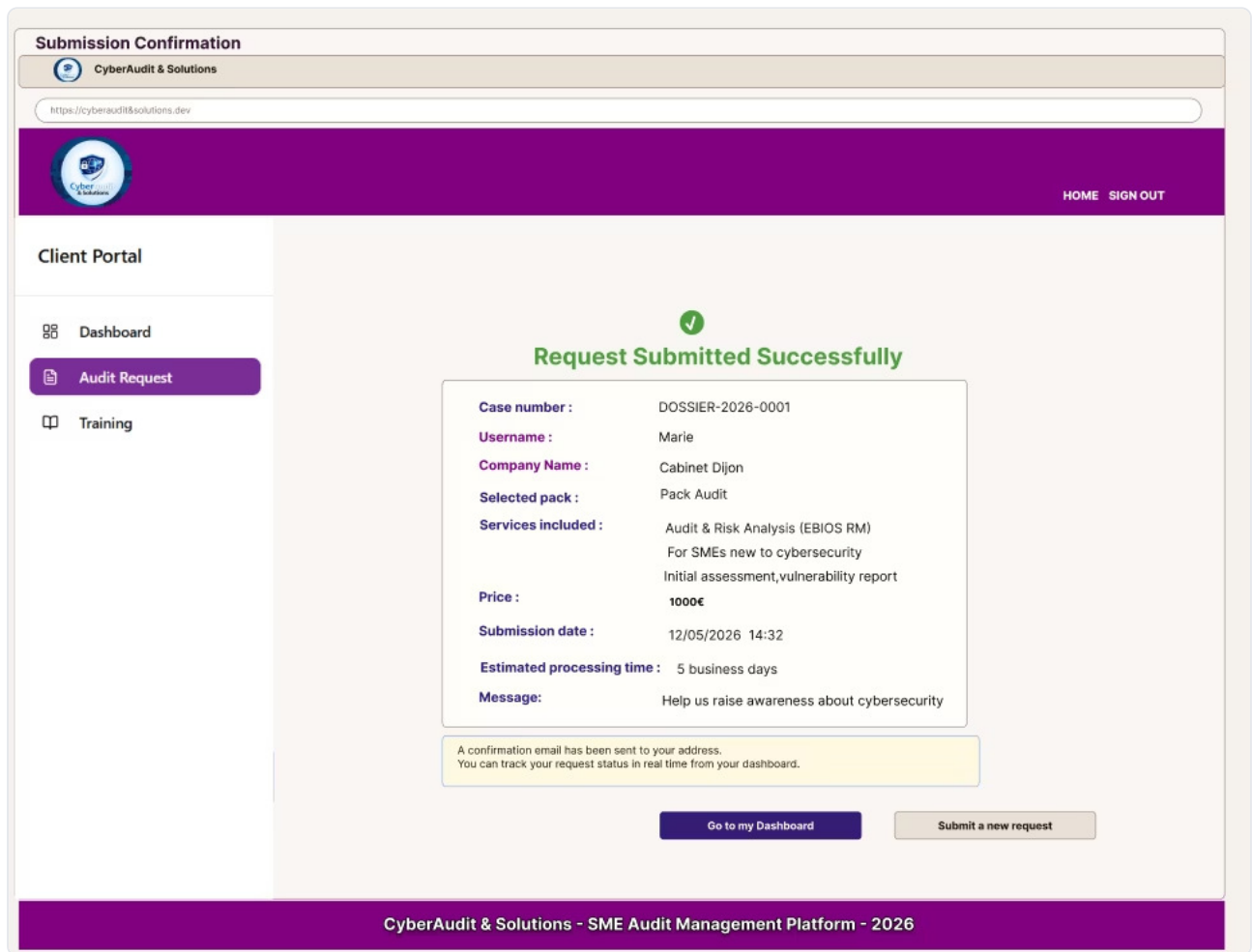
Sent the audit request

CyberAudit & Solutions - SME Audit Management Platform - 2026

Mockup 5 — Formulaire d'audit (3 étapes)

Écran 6 — Détail demande (client)

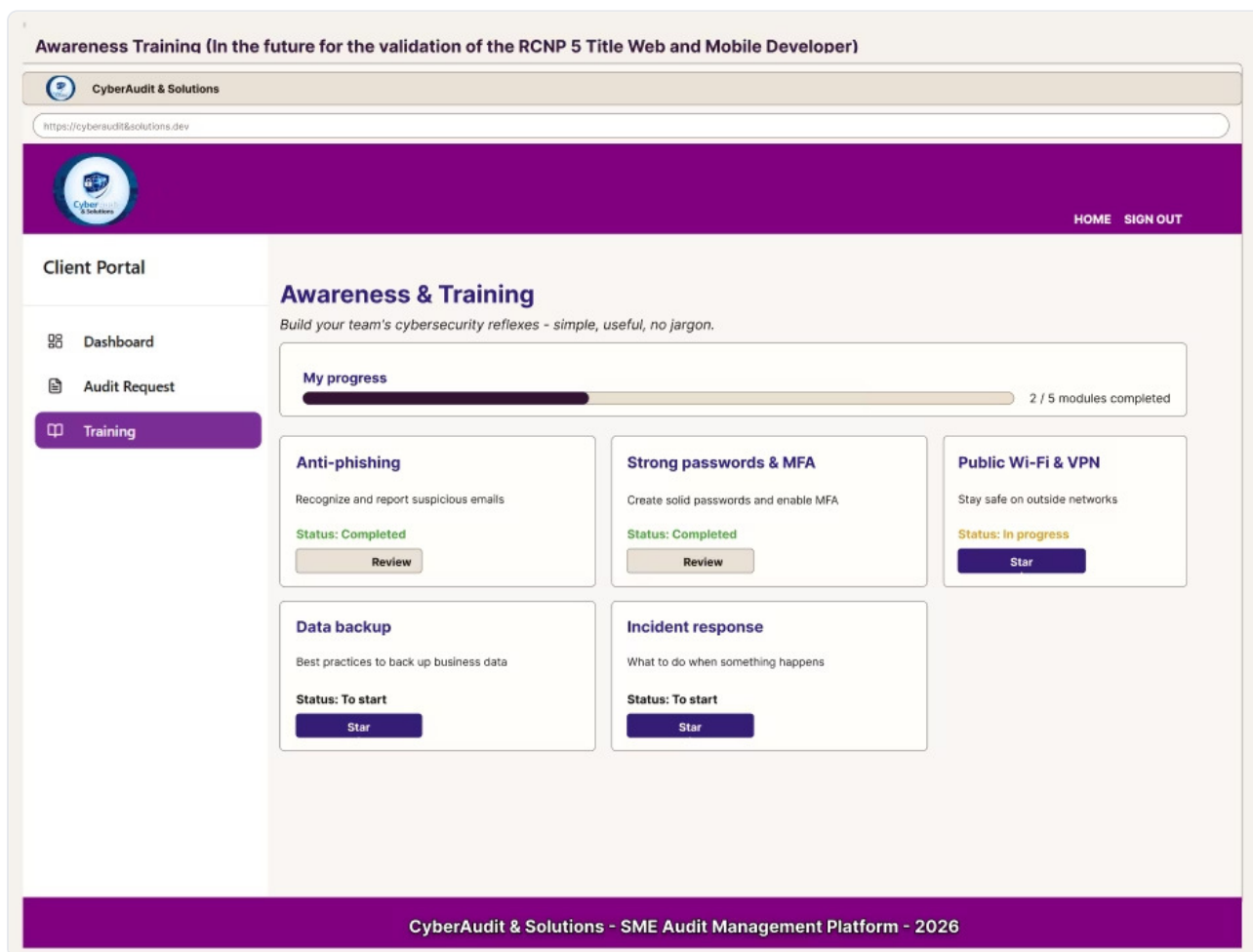
Statut, timeline. Après envoi, message de confirmation à l'expert (Karim) et accusé de réception à la cliente. La cliente suit l'avancée de l'analyse.



Mockup 6 — Détail demande côté client

Écran 7 — Module formations

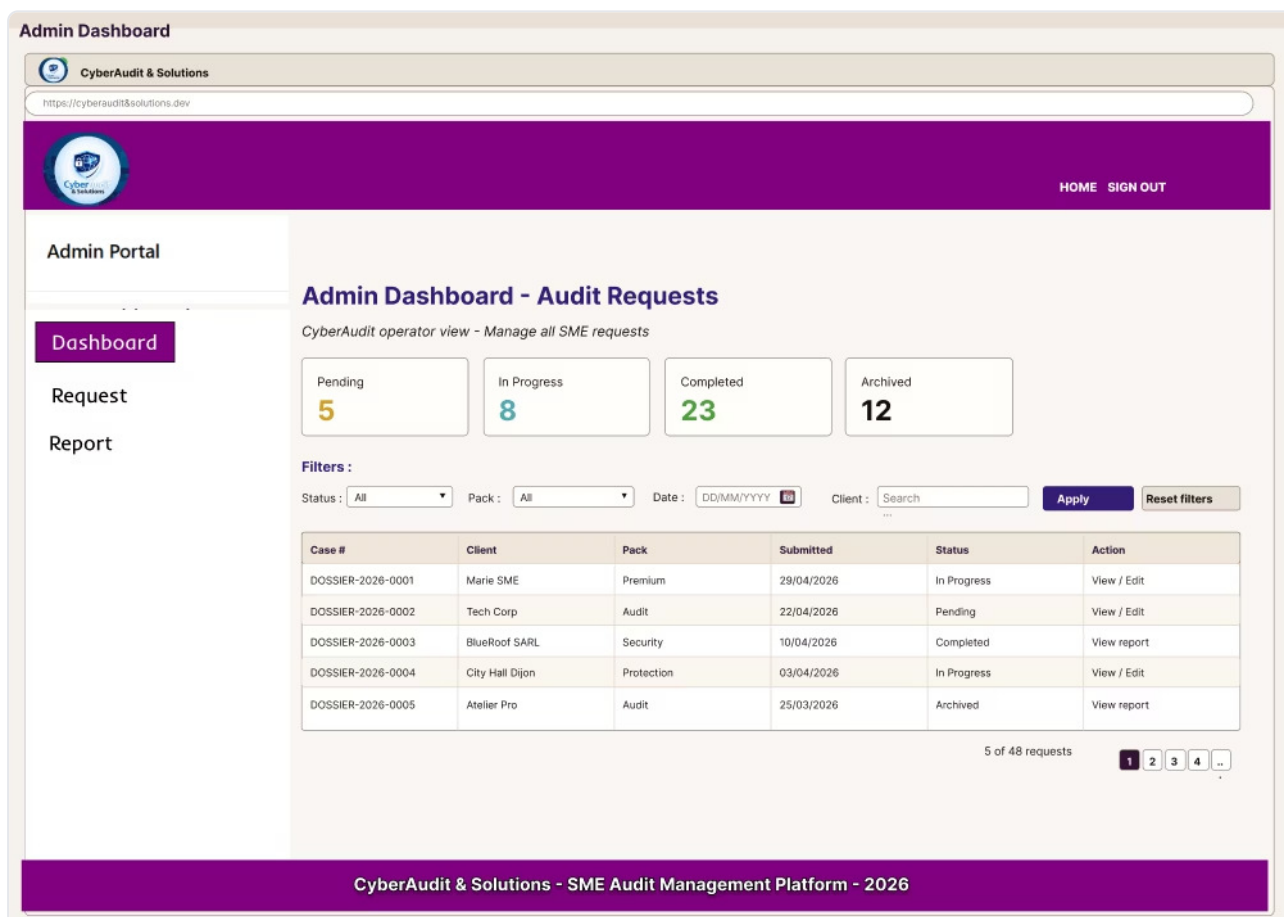
Liste cartes + détail markdown. (À développer ultérieurement). Modules : « anti-phishing », « Authentification à facteurs multiples », « Connexion publique et Wi-Fi & VPN », « Sauvegarde de données », « Réponse d'incidents ». Sous différents formats (vidéo, visio, documentation) avec barre de progression.



Mockup 7 — Modules de formation

Écran 8 — Dashboard Admin

Tableau + filtres. L'expert (Karim) accède à son tableau de bord avec toutes les demandes. Il peut trier la liste selon le pack choisi et les états actuels, puis consulter et éditer chaque demande.



Mockup 8 — Dashboard Admin

Écran 9 — Détail demande (admin)

Sélecteur statut + bouton générer. En cliquant sur « View/edit », Karim change le statut via le formulaire « Status & Actions », génère un rapport PDF via « Generate PDF report », envoie une notification au client (« Send notification ») et peut archiver la demande (« Archive request »).

Admin Request Detail

CyberAudit & Solutions

https://cyberauditsolutions.dev

CyberAudit & Solutions

HOME SIGN OUT

Admin Portal

Dashboard

Request

Report

Request DOSSIER-2026-0001

Back to list

Client information

Company : Marie SME (Cabinet Comptable)

Contact : Marie Dupont

Email : marie@cabinet-dijon.fr

Submitted : 29/04/2026 at 14:32

Pack selected : Pack Audit

Client message : Help us raise awareness about cybersecurity

Status & Actions

Current status : In Progress

Assigned to : Karim

Internal notes

EBIOS RM analysis ongoing. Awaiting client log files.

Save changes

Generate PDF report

Send notification

Archive request

History (immutable - GDPR)

Date	Author	Action
29/04/2026 14:32	Marie Dupont	Request submitted (Pack Premium)
29/04/2026 14:32	System	Confirmation email sent (Celery)
29/04/2026 16:10	Karim	Status changed: Pending -> In Progress
29/04/2026 16:11	Karim	Internal note added

CyberAudit & Solutions - SME Audit Management Platform - 2026

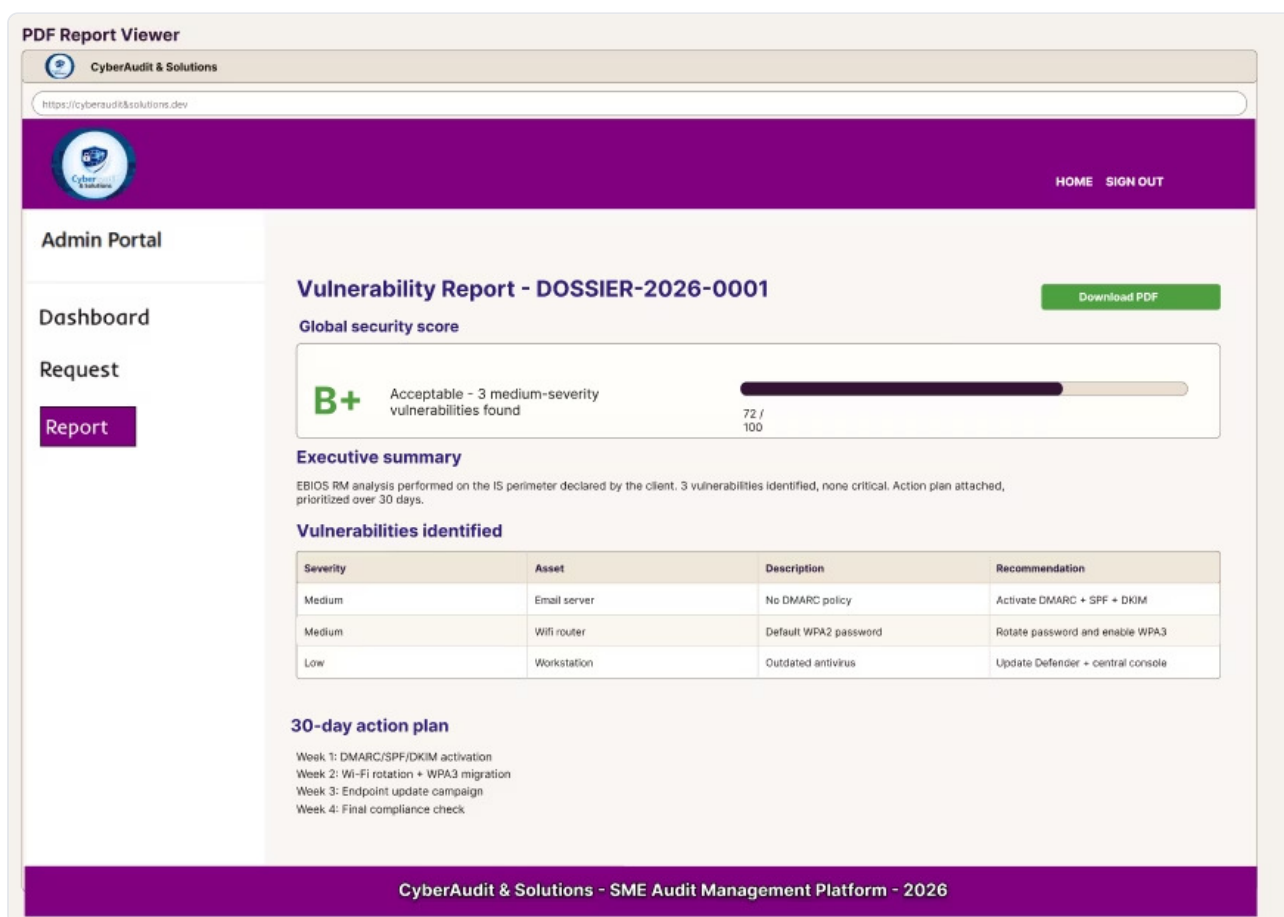
Mockup 9 — Détail demande côté admin

Écran 10 — Visionneuse de rapports PDF

Rapport peut être téléchargé. Le rapport identifie :

- Le niveau de vulnérabilités (Low, Medium, High)
- Les atouts (serveur mail, routeur wifi, etc.)
- La description
- Les recommandations

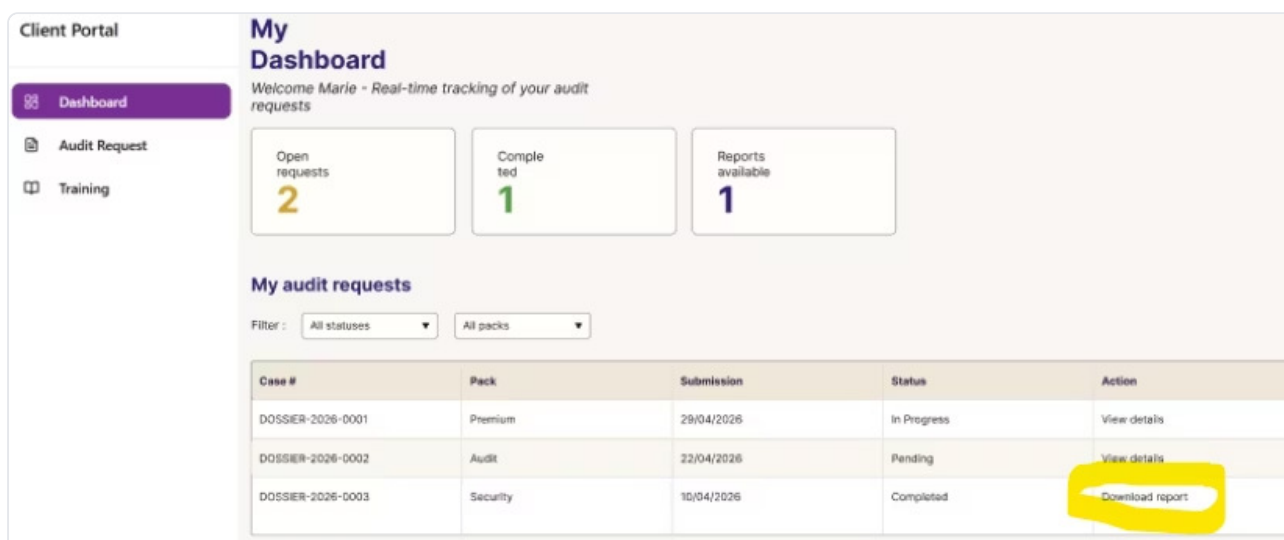
Un plan d'action est généré avec le nombre de jours estimé.



Mockup 10 — Visionneuse de rapport PDF

Écran 11 — Téléchargement depuis le dashboard

Le client peut télécharger le rapport depuis son tableau de bord via le lien « Download report ».



Mockup 11 — Téléchargement depuis le dashboard

3. Architecture du système

3.1 Vue d'ensemble — Architecture en couches

L'architecture suit un modèle en couches : Client (navigateur) → CDN/Backend → Données + Services externes.

- **Client (Navigateur)** : React SPA · Tailwind CSS · DOMPurify (anti-XSS), HTTPS, REST + JWT
- **Vercel / CDN** : Assets statiques · HTTPS Let's Encrypt
- **Back-End (Railway / Render)** : Django REST API (JSON, JWT, HTTPS) — Celery Worker (async) — Redis (broker) — ORM — SMTP
- **Couche données** : PostgreSQL 15 + Storage PDFs
- **Services externes** : SMTP Gmail (notifications email)

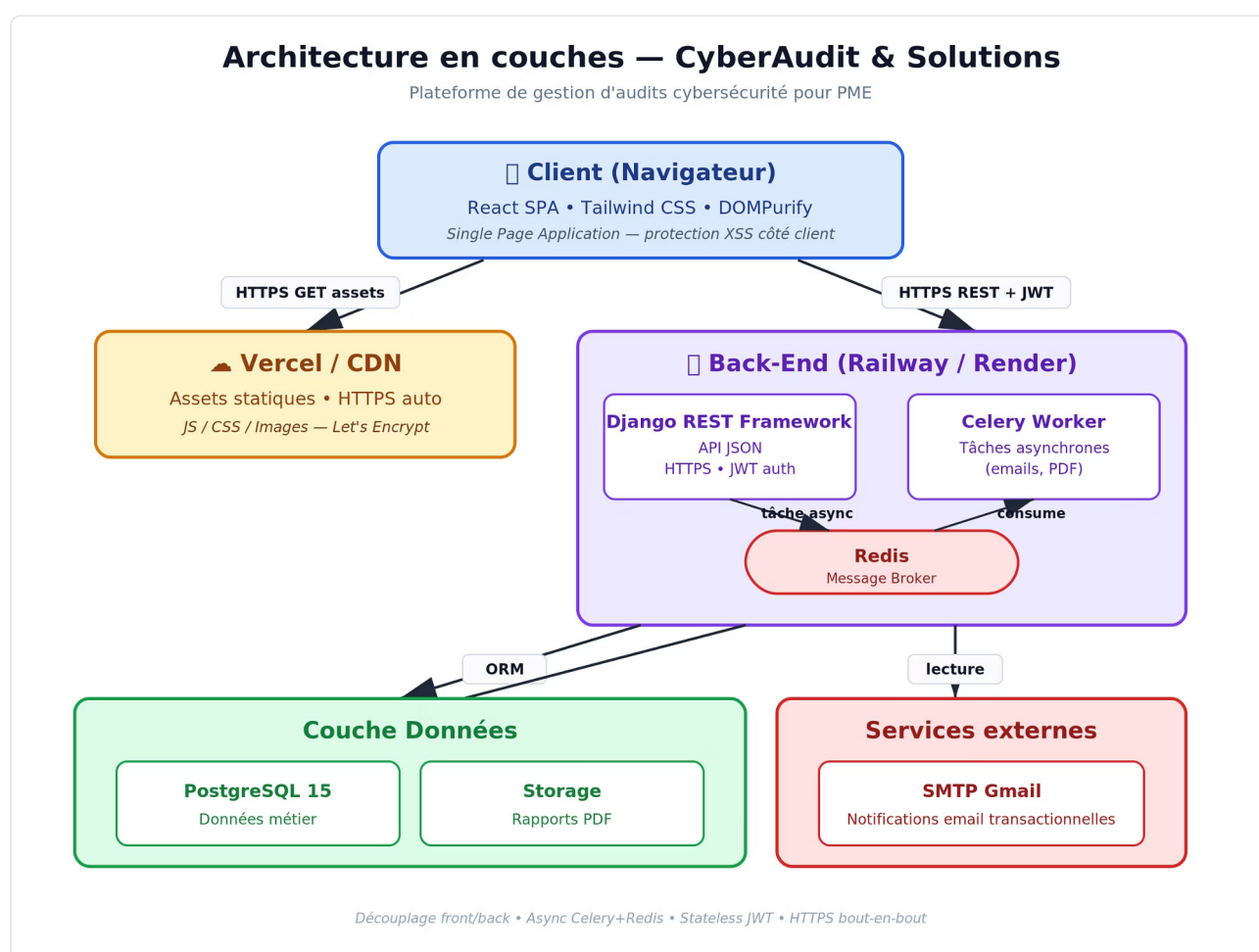


Figure 1 — Architecture en couches : Client → CDN/Backend → Données + Services externes

3.2 Flux de données — Demande d'audit

1. Marie remplit le formulaire dans la SPA React.
2. Le front envoie `POST /api/audits/` (JSON + JWT dans `Authorization: Bearer`).
3. Django valide la requête (DRF serializer) et persiste en BDD via l'ORM.
4. Django publie une tâche Celery `send_acknowledgment_email(audit_id)` dans Redis.

5. Django répond `201 Created` avec le détail de l'audit.
6. Karim (admin) voit la nouvelle demande dans son dashboard (polling 30 s — V2 : WebSocket).

3.3 Choix d'architecture justifiés

- **Découplage front/back** → 2 déploiements indépendants, CORS contrôlé, SPA cacheable sur CDN.
- **Async Celery** → la génération d'un PDF peut prendre 5-10 s ; on ne bloque pas le thread HTTP.
- **PostgreSQL relationnel** → données fortement structurées avec relations (User → AuditRequest → Report), intégrité ACID critique.
- **JWT stateless** → pas de session serveur, scalable horizontalement.

3.4 Accessibilité (WCAG 2.1 niveau AA)

La cible utilisatrice prioritaire (Marie, 45 ans, non-IT) impose un parcours accessible. La plateforme vise la conformité **WCAG 2.1 niveau AA** sur les 4 principes : perceptible, utilisable, compréhensible, robuste.

3.4.1 Règles appliquées

Principe WCAG	Mise en œuvre
Contraste minimal	Ratio 4.5:1 sur le texte normal, 3:1 sur le texte large. Palette <code>cyber-700</code> (<code>#5b21b6</code>) sur fond blanc validée — outil de vérification : <code>axe DevTools</code> .
Navigation clavier	Tous les éléments interactifs accessibles via <code>Tab</code> / <code>Shift+Tab</code> . Ordre logique de focus respectant la lecture. <code>:focus-visible</code> Tailwind avec anneau violet 2px sur chaque CTA.
ARIA labels	<code>StatusBadge</code> expose <code>aria-label="Statut : en cours"</code> . <code>SecurityScoreGauge</code> expose <code>role="meter"</code> + <code>aria-valuenow</code> . Icônes décoratives en <code>aria-hidden="true"</code> .
Formulaires	Chaque <code><input></code> a un <code><label></code> associé (pas seulement un placeholder). Messages d'erreur reliés via <code>aria-describedby</code> . Champs requis annoncés avec <code>aria-required</code> .
Structure sémantique	Balises <code><header></code> , <code><nav></code> , <code><main></code> , <code><footer></code> , hiérarchie de titres <code>h1</code> → <code>h6</code> sans saut. Skip-link "Aller au contenu" en première position.
Images et icônes	Attribut <code>alt</code> systématique : descriptif si informatif, vide si décoratif. Diagrammes d'architecture proposés en SVG accessible + description textuelle équivalente.
Multilingue	Attribut <code>lang="fr"</code> sur <code><html></code> . Aucun contenu mélangeant FR/EN sans changement de <code>lang</code> .
Animations	Respect de <code>prefers-reduced-motion</code> via media query Tailwind : transitions désactivées si l'utilisateur le demande dans son OS.

3.4.2 Tests d'accessibilité automatisés

- **axe-core** intégré aux tests Vitest : `npm install -D @axe-core/react jest-axe`. Chaque composant majeur (LoginForm, AuditRequestForm, RequestTable) testé via `expect(await axe(container)).toHaveNoViolations()`.
- **Lighthouse CI** dans le pipeline GitHub Actions : score Accessibilité ≥ 90 obligatoire pour merge.
- **Tests manuels W7-W10** : navigation 100 % clavier sur les 3 parcours critiques (inscription, soumission audit, téléchargement PDF), test lecteur d'écran avec NVDA (Windows) et VoiceOver (macOS).

3.4.3 Cible mesurable

Score Lighthouse Accessibilité $\geq 90/100$ sur les 4 pages principales (landing, login, dashboard client, formulaire audit) avant la soutenance W12.

3.5 Budget de performance (Web Vitals)

L'expérience utilisatrice cible (Marie, parcours < 3 min) impose une exigence de performance non négociable. La plateforme se fixe un **budget de performance** aligné sur les *Core Web Vitals* de Google.

3.5.1 Cibles Web Vitals

Métrique	Cible "Good"	Définition & impact
LCP Largest Contentful Paint	$< 2,5$ s	Délai d'affichage du plus gros élément visible. Mesure la perception de chargement.
INP Interaction to Next Paint	< 200 ms	Latence de réponse aux interactions utilisateur (remplace FID depuis mars 2024).
CLS Cumulative Layout Shift	$< 0,1$	Stabilité visuelle — pas de saut de mise en page intempestif.
TTI Time to Interactive	$< 3,5$ s	Délai avant qu'on puisse cliquer / taper sans lag.
TTFB Time to First Byte	< 600 ms	Délai serveur — couvre Django + base de données.
Bundle JS	< 200 kB gzipped	Poids du JavaScript initial à charger. Au-delà, dégradation 3G/4G.
Lighthouse Perf	$\geq 85/100$	Score global agrégé, calculé en CI à chaque PR.

3.5.2 Stratégies d'optimisation

- **Code splitting** via `React.lazy()` et `Suspense` sur chaque route (`LoginPage`, `ClientDashboard`, `AdminDashboard`, `TrainingPage`) → chargement initial allégé.
- **Tree-shaking** automatique par Vite, imports nommés uniquement (pas de `import * as ...`).
- **Images** : `<img loading="lazy">` par défaut, format AVIF/WebP via `vite-plugin-imagemin`, dimensions explicites (anti-CLS).
- **Polices web** : `font-display: swap`, préchargement de la police principale via `<link rel="preload">`.
- **Cache HTTP** : assets statiques Vercel avec `Cache-Control: public, max-age=31536000, immutable` (cache busting par hash).
- **API** : pagination 20 items par défaut, jamais de `SELECT *` dans Django (`only()` / `defer()` sur les listes).
- **BDD** : index BTREE et composites (cf. §4.1), `select_related` / `prefetch_related` systématiques pour éviter les N+1.

3.5.3 Mesure et CI

- **Lighthouse CI** dans GitHub Actions : exécution sur chaque PR sur les 4 pages clés, échec si score Performance < 85 ou si Web Vitals dépassent les cibles.
- **Sentry Performance** : suivi p50/p75/p95 du LCP et de l'INP en production (10 % de sample rate).
- **Bundle analyzer** : `rollup-plugin-visualizer` exécuté à chaque release, alerte si le bundle dépasse 200 kB.
- **Real User Monitoring (V2)** : `web-vitals.js` envoie les métriques réelles vers un endpoint `/api/metrics/vitals/`.

4. Composants, classes et BDD

4.1 Schéma BDD

Entité	Champs clés	Relations
USER	id (UUID), email (UK), password_hash, first_name, last_name, role (client/admin), company_name, created_at, is_active	1→N AuditRequest (submits / follows / receives)
AUDIT_PACK	id, reference, code (audit/security/protection/premium), name, description, duration_days, price	1→N AuditRequest (chosen_by)
AUDIT_REQUEST	id (UUID), client_id (FK), pack_id (FK), status (pending/in_progress/done/archived), scope_notes, submitted_at, updated_at, completed_at	N→1 User+Pack ; 1→1 Report ; 1→N Notification
AUDIT_REPORT	id (UUID), audit_request_id (FK UK), pdf_path, summary, findings (JSON), generated_at	1→1 AuditRequest
TRAINING_MODULE	id, title, content_md, duration_min, level, published_at	1→N Progress
TRAINING_PROGRESS	id, user_id, module_id, completed, started_at, completed_at	N→1 User + Module
NOTIFICATION	id, user_id, request_id, type, subject, status, sent_at	N→1 User + Request

Index recommandés :

- `USER.email` UNIQUE
- `AUDIT_REQUEST.status` BTREE (filtrage admin fréquent)
- (`AUDIT_REQUEST.client_id` , `AUDIT_REQUEST.created_at`) composé (dashboard client)
- `AUDIT_REPORT.audit_request_id` UNIQUE (1 rapport par demande)

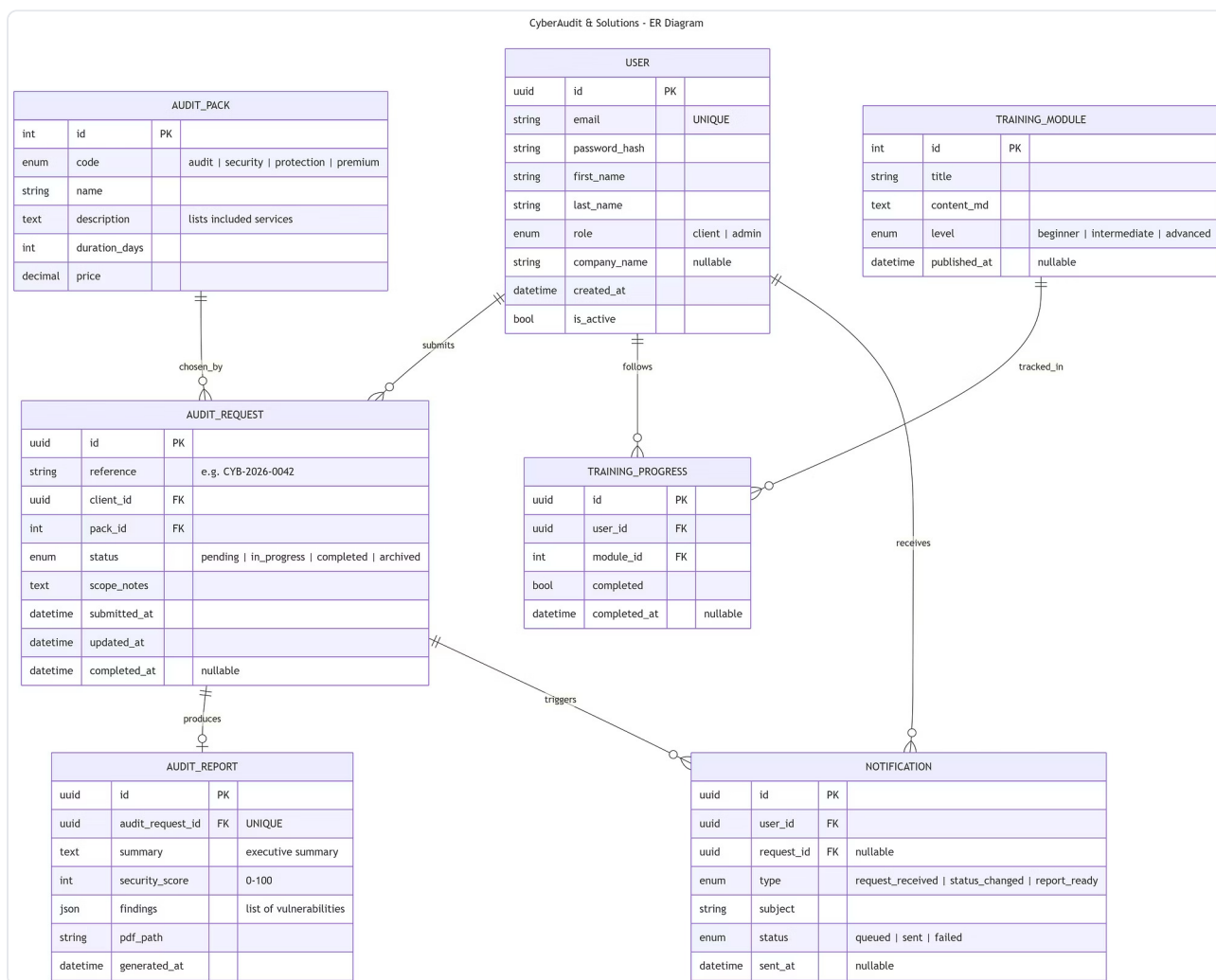


Figure 2 — Schéma relationnel de la base de données

4.2 Diagramme de classes back-end (Django models + services)

Classe User

Représente un utilisateur de la plateforme, qu'il soit client SME ou administrateur. Gère l'authentification JWT et la séparation des rôles.

Attribut	Description
<code>id : UUID</code>	Identifiant unique (PK)
<code>email : str</code>	Adresse e-mail (unique, indexée)
<code>_password_hash : str</code>	Mot de passe haché (bcrypt / PBKDF2)
<code>first_name, last_name</code>	Prénom et nom
<code>role : enum</code>	client ou admin
<code>company_name : str</code>	Nom de l'entreprise (nullable)
<code>is_active : bool</code>	Compte activé ou désactivé
<code>created_at : datetime</code>	Date de création du compte

Méthodes : `check_password(raw)`, `has_role(role)`

Classe AuditPack

Représente l'un des 4 packs (Audit, Security, Protection, Premium). Données de référence chargées via fixture.

<code>id : int</code>	Identifiant unique
<code>code : enum</code>	audit / security / protection / premium
<code>name : str</code>	Nom commercial
<code>description : str</code>	Liste des services inclus
<code>duration_days : int</code>	Délai estimé de traitement
<code>price : Decimal</code>	Tarif du pack

Classe AuditRequest

Cœur du métier : la demande d'audit soumise par un client SME. Cycle de 4 statuts (pending → in_progress → completed → archived).

<code>id : UUID</code>	Identifiant unique
<code>reference : str</code>	Numéro de dossier (ex : CYB-2026-0042)
<code>client_id : FK → User</code>	Client SME ayant soumis
<code>pack_id : FK → AuditPack</code>	Pack choisi
<code>status : enum</code>	pending / in_progress / completed / archived
<code>scope_notes : str</code>	Précisions du formulaire
<code>submitted_at, completed_at</code>	Dates de cycle

Méthodes : `generate_reference()`, `update_status(new_status)`, `estimated_completion()`

Classe AuditReport

Rapport de vulnérabilités lié à une seule AuditRequest, inclut un score visuel et un PDF téléchargeable.

<code>id : UUID</code>	Identifiant unique
<code>audit_request_id : FK</code>	Relation 1-1
<code>summary : str</code>	Résumé exécutif vulgarisé
<code>security_score : int</code>	Score 0-100 (jauge)
<code>findings : JSON</code>	Liste des vulnérabilités
<code>pdf_path : str</code>	Chemin du PDF généré
<code>generated_at</code>	Date de génération

Méthodes : `build_pdf()`, `get_download_url()`

Classe Notification

Trace toutes les communications e-mail envoyées via Celery.

id : UUID	Identifiant unique
user_id : FK → User	Destinataire
request_id : FK	Demande concernée (nullable)
type : enum	request_received / status_changed / report_ready
status : enum	queued / sent / failed

Méthodes : send(), retry()

Classes TrainingModule & TrainingProgress

Modules de sensibilisation cybersécurité (phishing, MFA, RGPD...) avec suivi de progression utilisateur.

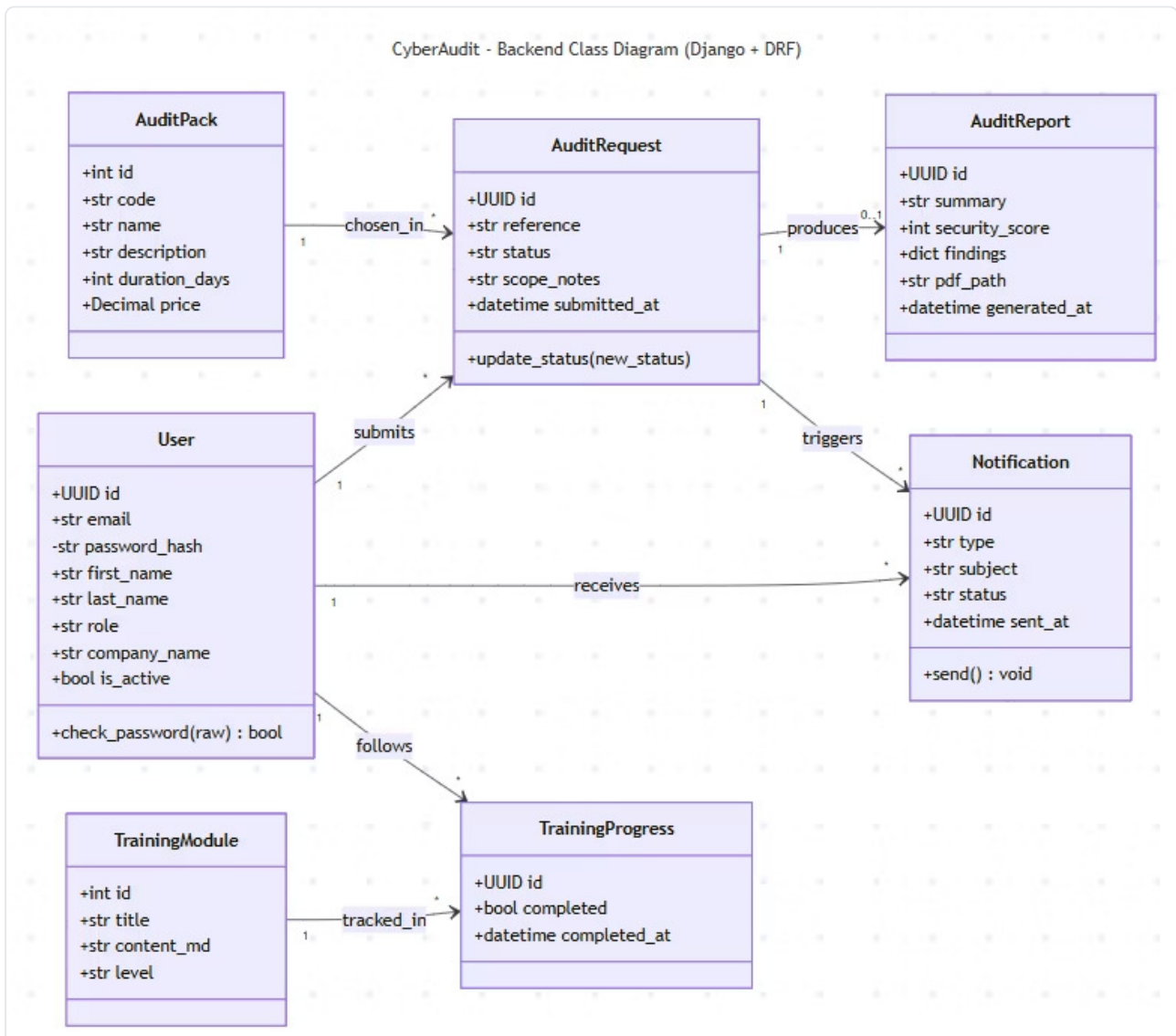


Figure 3 — Diagramme de classes back-end Django

Exemple — `accounts/models.py`

```
class User(AbstractBaseUser, PermissionsMixin):
    """Utilisateur de la plateforme – client PME ou admin CyberAudit."""
    id = UUIDField(primary_key=True, default=uuid4)
    email = EmailField(unique=True, db_index=True)
    role = CharField(choices=[('client', 'Client'), ('admin', 'Admin')])
    company_name = CharField(max_length=200, blank=True)
    company_size = CharField(choices=[('TPE', 'TPE'), ('PME', 'PME'), ('ETI', 'ETI')])
    sector = CharField(max_length=100, blank=True)
    is_active = BooleanField(default=True)
    created_at = DateTimeField(auto_now_add=True)
    last_login = DateTimeField(null=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

    def is_admin(self) -> bool: ...
    def is_client(self) -> bool: ...
```

Exemple — `audits/models.py`

```
class AuditPack(models.Model):
    """Catalogue des 4 packs d'audit proposés."""
    name = CharField(max_length=50)
    description = TextField()
    duration_days = PositiveIntegerField()
    price = DecimalField(max_digits=8, decimal_places=2)

class AuditRequest(models.Model):
    """Demande d'audit soumise par un client."""
    STATUS_CHOICES = [
        ('pending', 'En attente'),
        ('in_progress', 'En cours'),
        ('done', 'Terminé'),
        ('archived', 'Archivé'),
    ]
    id = UUIDField(primary_key=True, default=uuid4)
    client = ForeignKey(User, on_delete=PROTECT, related_name='audit_requests')
    pack = ForeignKey(AuditPack, on_delete=PROTECT)
    status = CharField(choices=STATUS_CHOICES, default='pending',
db_index=True)
    scope_notes = TextField(blank=True)
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
    completed_at = DateTimeField(null=True, blank=True)

    def transition_to(self, new_status: str) -> None:
        """Change le statut, met à jour completed_at et déclenche la notification."""
        ...
```

```
def can_generate_report(self) -> bool:
    return self.status == 'done'
```

Services métier (logique séparée des modèles)

```
# audits/services.py
class AuditRequestService:
    @staticmethod
    def create_request(client: User, pack_id: int, scope: str) -> AuditRequest: ...

    @staticmethod
    def change_status(request: AuditRequest, new_status: str, by: User) -> None:
        """Change le statut + déclenche email Celery."""
        ...

# reports/services.py
class PDFReportGenerator:
    @staticmethod
    def generate(audit_request: AuditRequest) -> AuditReport:
        """Génère un PDF avec WeasyPrint, stocke le chemin, persiste."""
        ...

# notifications/tasks.py (Celery)
@shared_task
def send_status_change_email(audit_id: str, new_status: str) -> None: ...
```

4.3 Composants front-end (React + Tailwind CSS)

Le front-end est une SPA React communiquant avec l'API REST via JWT. L'arborescence se structure en pages (vues complètes), composants UI réutilisables, store global et services d'appel API.

Authentification & routage

AuthStore — State global : `user`, `accessToken`, `refreshToken`. Méthodes : `login`, `logout`, `refresh`, `hasRole`.

ProtectedRoute — Composant de garde, props : `allowedRoles`. Redirige vers `/login` si non authentifié.

ApiClient — Wrapper axios qui ajoute automatiquement le JWT à chaque requête et gère le refresh token en cas de 401.

Pages publiques

LoginPage — Formulaire de connexion. Soumet à `AuthStore.login()` puis redirige selon le rôle.

RegisterPage — Formulaire d'inscription client SME. Appelle `AuthStore.register()`.

Espace client SME

ClientDashboard — Tableau de bord listant les demandes du client connecté.

AuditRequestForm — Formulaire en 3 étapes (pack, périmètre, confirmation). Affiche 4 PackCard.

RequestDetail — Vue détaillée avec timeline, rapport PDF, score. Composants : `StatusBadge`, `SecurityScoreGauge`, `FindingsList`.

Espace administrateur CyberAudit

AdminDashboard — Vue paginée de toutes les demandes avec filtres (statut, pack, date, client) et tri.

Module de formation

TrainingPage — Liste des modules avec indicateur de progression.

TrainingModuleViewer — Affiche le contenu Markdown. Sécurité : nettoyage via DOMPurify (anti-XSS, SMART #3).

Composants UI réutilisables

Navbar, **PackCard**, **StatusBadge** (gris/bleu/vert/violet), **SecurityScoreGauge** (jauge circulaire, rouge < 40, orange 40-70, vert > 70), **RequestTable**, **Pagination**, **ToastContainer**.

Flux d'interaction principal — Soumission demande

1. L'utilisateur se connecte via LoginPage → AuthStore stocke le JWT.
2. ProtectedRoute autorise l'accès au ClientDashboard.
3. Clic sur « Nouvelle demande » → AuditRequestForm charge les packs.
4. Sélection d'une PackCard → AuditService.create() appelle l'API Django.
5. Le back-end crée l'AuditRequest et déclenche la tâche Celery d'email.
6. Le client est redirigé vers RequestDetail (StatusBadge "pending").
7. À chaque changement de statut côté admin, une Notification est envoyée.

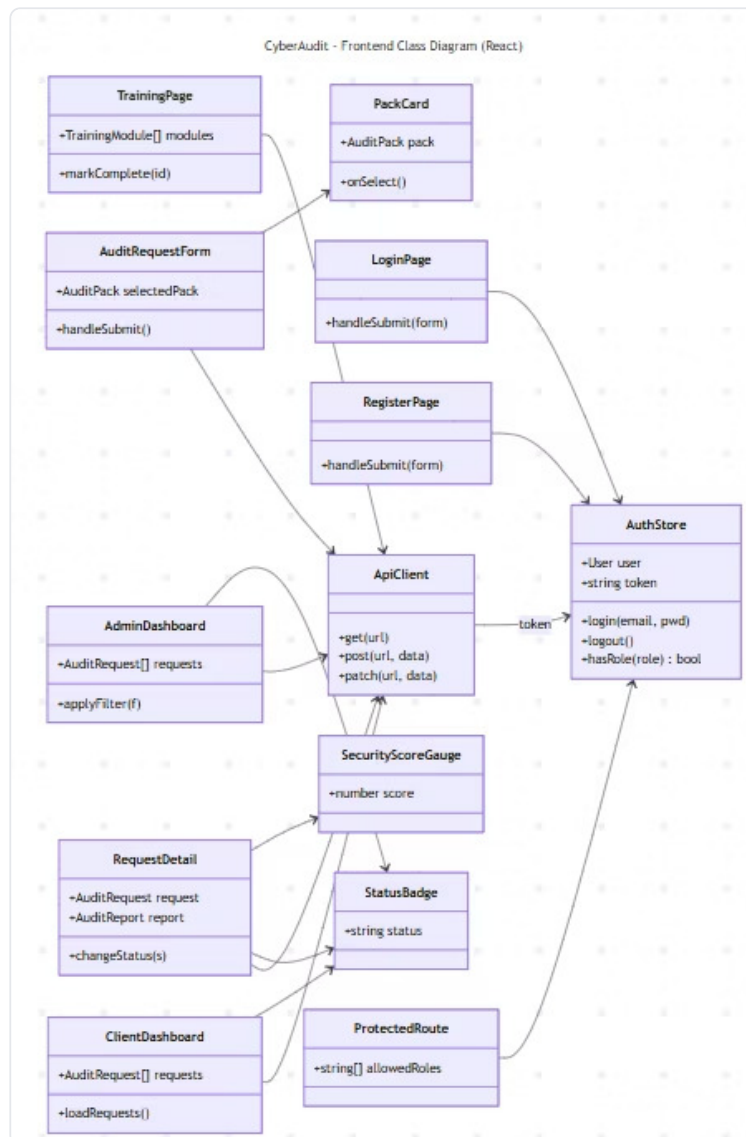


Figure 4 — Arborescence des composants React

Arborescence des composants

```

src/
├── components/
│   ├── auth/           LoginForm, RegisterForm, ProtectedRoute
│   ├── dashboard/      ClientDashboard, AdminDashboard, AuditRequestCard, StatusBadge
│   ├── audit/          AuditRequestForm, PackSelector, AuditDetailView
│   ├── training/       ModuleList, ModuleViewer
│   └── shared/         Header, Sidebar, Button, ErrorBoundary
├── pages/              HomePage, LoginPage, RegisterPage, DashboardPage,
                        NewAuditPage, AuditDetailPage, TrainingPage
├── hooks/              useAuth, useAudits, useApi
├── services/           api.js (axios + JWT), auth/audits/reports services
├── utils/              sanitize.js (DOMPurify), validators.js
└── App.jsx             routing global
  
```

Conventions : Composants fonctionnels + hooks (pas de classes), état global via Context API, sanitize() sur tous les inputs.

4.4 Génération du rapport PDF de vulnérabilités

Le rapport PDF est le livrable principal du client SME : il consolide l'analyse de vulnérabilités, le score de sécurité et le plan d'action recommandé. Sa génération est entièrement asynchrone pour ne pas bloquer l'interface admin.

Stack technique

- **WeasyPrint** : Convertit un template HTML+CSS en PDF de qualité impression. Réutilise la charte graphique du web.
- **Celery** : Exécute la tâche `generate_pdf_task` en arrière-plan.
- **Redis** : File d'attente Celery (scaling parallèle).
- **Stockage** : `/media/reports/` au MVP, S3 en V2 ; accès via URL signée 15 min.

Structure du PDF généré

Section	Contenu
Page 1 — Couverture	Logo CyberAudit, nom client, référence dossier (CYB-2026-0042), date, confidentialité
Page 2 — Synthèse exécutive	Score global A à F (jauge 0-100) + résumé en 3 lignes vulgarisées
Page 3 — Périmètre audité	Liste des actifs analysés (serveur mail, routeur Wi-Fi, postes...) + méthodologie EBIOS RM
Pages 4-N — Vulnérabilités	Tableau findings : sévérité (Critical/High/Medium/Low), actif, description, recommandation
Page finale — Plan d'action 30 jours	Étapes hebdomadaires (S1, S2, S3, S4) avec actions concrètes et priorités
Annexe	Glossaire, références (ISO 27005, ANSSI), contact CyberAudit

Flux complet — de la demande au téléchargement

1. L'admin clique sur « Generate PDF report » dans `AdminRequestDetail`.
2. Le front appelle `POST /api/requests/{id}/generate-report` avec les findings.
3. Le `PdfReportService` Django crée l'enregistrement `AuditReport` (résumé, score, findings JSON).
4. Il enfile la tâche `generate_pdf_task.delay(report_id)` et retourne `202 Accepted`.
5. Le worker Celery rend `report.html` avec les données, puis WeasyPrint produit le PDF.
6. Fichier sauvegardé dans `/media/reports/CYB-2026-0042.pdf`, `pdf_path` mis à jour.
7. Notification de type `report_ready` créée, e-mail envoyé via `send_email_task`.
8. Côté client, `ClientDashboard` affiche « Download report » → `GET /api/reports/{id}/download` en streaming.

Sécurité d'accès au PDF

- **JWT obligatoire** sur `/api/reports/{id}/download`. Aucun accès anonyme.
- **Permission** `IsAdminOrOwner` : 403 Forbidden sinon.
- **URL signée** avec timestamp, expire après 15 minutes.
- **Pas de cache navigateur** : `Cache-Control: no-store, private`.

- **Audit log** (V2) : qui, quand, depuis quelle IP — conforme RGPD.

Gestion des échecs de génération

- **Retry automatique** : `autoretry_for=(Exception,)` , `max_retries=3` , backoff exponentiel (10s, 30s, 90s).
- **Statut intermédiaire** : `pdf_path` vide tant que non généré → front affiche « Génération en cours... ».
- **Notification d'échec** : après 3 échecs, type `generation_failed` envoyée à l'admin.
- **Régénération manuelle** via le bouton « Generate PDF report ».

4.5 Monitoring & observabilité

L'application doit rester traçable en production : on doit savoir *qui a fait quoi, quand, et pourquoi ça a planté*. La stratégie repose sur 3 piliers : logs structurés, monitoring d'erreurs, alerting.

4.5.1 Logs structurés (JSON)

Django configure `structlog` pour produire des logs JSON consommables par Railway et un futur ELK/Loki. Chaque log porte au minimum : `timestamp` , `level` , `request_id` (corrélation), `user_id` , `endpoint` , `latency_ms` .

```
{
  "timestamp": "2026-05-18T09:24:11Z",
  "level": "INFO",
  "event": "audit_request.created",
  "request_id": "req-7f2a...",
  "user_id": "8f2a...c9",
  "audit_id": "CYB-2026-0042",
  "pack": "security",
  "latency_ms": 142
}
```

4.5.2 Monitoring d'erreurs — Sentry

- **Back-end** : `sentry-sdk[django,celery]` capture toutes les exceptions Django et Celery avec stack trace, context utilisateur, payload de la requête (PII anonymisée).
- **Front-end** : `@sentry/react` capture les erreurs JS et les rejets de promesse non gérés, plus les sessions replay sur erreur.
- **Release tracking** : chaque tag git pousse une release Sentry → suivi de régression par version.
- **Sample rate** : 100 % des erreurs en MVP ; 10 % des transactions performance pour rester dans le quota gratuit.

4.5.3 Métriques business & techniques

Métrique	Source	Seuil d'alerte
Temps de réponse API (p95)	Sentry Performance / Railway metrics	> 800 ms pendant 5 min
Taux d'erreur 5xx	Sentry	> 1 % sur 10 min
Échecs Celery consécutifs	Logs structurés	≥ 3 sur la même tâche

Métrique	Source	Seuil d'alerte
Connexions BDD actives	Railway Postgres metrics	> 80 % du pool
Espace disque /media	Railway	> 85 %
Demandes d'audit / jour	Django + dashboard admin	Métrique de croissance, pas d'alerte

4.5.4 Alerting

- **Sentry** envoie un e-mail au référent technique sur toute nouvelle erreur non vue, et un Slack DM (V2) si taux d'erreur > seuil.
- **Healthcheck** : endpoint `GET /api/health/` vérifie BDD, Redis, SMTP. Surveillé par UptimeRobot (gratuit) toutes les 5 min.
- **Notification d'échec Celery** : après 3 retries, un e-mail admin est envoyé (déjà documenté en §4.4).

4.5.5 Audit log applicatif (V2)

Une table `AUDIT_LOG` trace les actions sensibles : connexions, téléchargement de rapport, changement de statut, suppression de compte. Champs : `id`, `user_id`, `action`, `resource`, `ip`, `user_agent`, `created_at`. Conservation 1 an, conforme RGPD (cf. §7.5).

4.6 Migrations BDD & seeding

Le passage d'une version de schéma à l'autre, et le pré-remplissage des données de référence (4 packs d'audit, modules de formation initiaux, compte admin) sont automatisés. Aucune intervention manuelle en production.

4.6.1 Stratégie de migrations Django

- **Convention** : 1 migration = 1 changement atomique. Nom explicite : `0007_add_security_score_to_auditreport.py`.
- **Schema migrations** : générées via `python manage.py makemigrations`, jamais éditées à la main.
- **Data migrations** : utilisent `migrations.RunPython(forward, reverse)` avec une fonction `reverse` systématique pour permettre le rollback.
- **Exécution en CI** : `python manage.py migrate --check` dans le pipeline pour détecter les migrations non générées.
- **Production** : Railway lance `migrate` automatiquement au déploiement via le `release command` du `Procfile`.

4.6.2 Exemple de data migration

```
# audits/migrations/0002_seed_packs.py
from django.db import migrations

def seed_packs(apps, schema_editor):
    AuditPack = apps.get_model('audits', 'AuditPack')
    AuditPack.objects.bulk_create([
        AuditPack(code='audit', name='Audit', duration_days=5, price=490),
```

```

        AuditPack(code='security', name='Security', duration_days=10, price=990),
        AuditPack(code='protection', name='Protection', duration_days=15,
price=1490),
        AuditPack(code='premium', name='Premium', duration_days=20,
price=2490),
    ])

def remove_packs(apps, schema_editor):
    apps.get_model('audits', 'AuditPack').objects.all().delete()

class Migration(migrations.Migration):
    dependencies = [('audits', '0001_initial')]
    operations = [migrations.RunPython(seed_packs, remove_packs)]

```

4.6.3 Fixtures (données de démo)

Des fixtures JSON dans `backend/fixtures/` permettent de charger des données de démo (5 clients fictifs, 10 demandes d'audit, 3 rapports) pour les démos jury et les tests E2E :

```

python manage.py loaddata fixtures/demo_clients.json
python manage.py loaddata fixtures/demo_audits.json
python manage.py loaddata fixtures/demo_reports.json

```

Un script wrapper `scripts/seed_demo.sh` enchaîne `migrate` + `loaddata` pour un environnement de démo en 1 commande.

4.6.4 Création de l'admin initial

Aucun mot de passe en clair dans le code. La commande Django personnalisée `create_initial_admin` lit `ADMIN_EMAIL` et `ADMIN_PASSWORD` depuis les variables d'environnement et crée l'utilisateur si la table est vide :

```

python manage.py create_initial_admin
# Idempotent : ne fait rien si un admin existe déjà

```

4.6.5 Stratégie de rollback

Scénario	Procédure
Migration cassée juste après déploiement	<code>python manage.py migrate audits 0006</code> revient à la migration précédente. La fonction <code>reverse</code> de <code>RunPython</code> est exécutée.
Données corrompues détectées	Restauration du backup Railway le plus récent (1 clic), puis investigation hors prod sur snapshot.
Migration nécessitant downtime	Maintenance window planifiée, page de maintenance activée via Vercel, migration exécutée, vérification, réouverture.
Changement de schéma non rétro-compatible	Stratégie « expand & contract » : 1) ajouter la nouvelle colonne, 2) double-écriture côté app, 3) backfill, 4) basculer la lecture, 5) supprimer l'ancienne colonne.

4.6.6 Snapshot avant chaque release

Avant chaque merge sur `main` (release prod), un snapshot manuel de la BDD Railway est déclenché via `railway db snapshot create`. Conservation 14 jours. Permet un rollback < 5 min en cas de problème majeur.

5. Diagrammes de séquence

5.1 Inscription + Connexion JWT

L'utilisateur saisit son email et son mot de passe. Le front appelle `POST /api/auth/register/`, puis `POST /api/auth/login/`. Django valide les credentials, génère access + refresh tokens, et retourne le tout au front. Le front stocke les tokens dans AuthStore et redirige vers le dashboard correspondant au rôle.

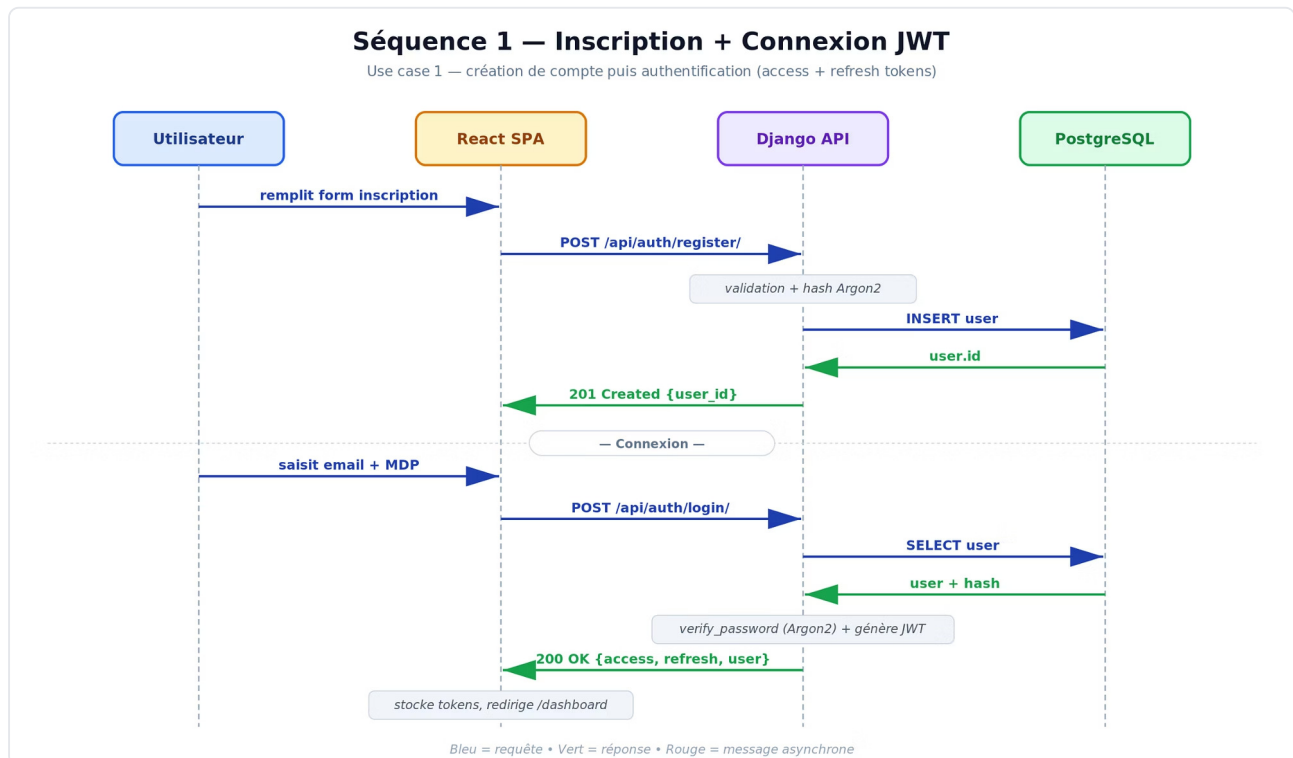


Figure 5 — Séquence Inscription + Connexion JWT

Messages échangés (en détail)

1. **Marie** → **RegisterPage** : remplit le formulaire (email, password, company_name).
2. **RegisterPage** → **ApiClient** : `POST /api/auth/register/` avec body JSON `{email, password, company_name, company_size, sector}`.
3. **ApiClient** → **Django (RegisterView)** : DRF serializer valide les champs (email unique, password robuste).
4. **Django** → **PostgreSQL** : `INSERT INTO accounts_user` avec password haché bcrypt.
5. **Django** → **Celery** : `send_welcome_email.delay(user_id)`.
6. **Django** → **RegisterPage** : `201 Created` avec `{user, access, refresh}`.
7. **RegisterPage** → **AuthStore** : stocke `accessToken` et `refreshToken`.
8. **RegisterPage** → **React Router** : redirection vers `/dashboard` selon `user.role`.
9. **Celery worker** → **SMTP Gmail** : envoie l'e-mail de bienvenue (asynchrone, hors thread HTTP).

Mermaid (rendu sur GitHub)

```
sequenceDiagram
    actor Marie
    participant Front as RegisterPage
    participant API as Django /auth
    participant DB as PostgreSQL
    participant Celery
    participant SMTP as Gmail SMTP

    Marie->>Front: Remplit formulaire
    Front->>API: POST /auth/register/ {email, password, ...}
    API->>API: Valider (DRF serializer)
    API->>DB: INSERT user (password bcrypt)
    API->>Celery: send_welcome_email.delay(user_id)
    API-->>Front: 201 {user, access, refresh}
    Front->>Front: AuthStore.set(tokens)
    Front-->>Marie: Redirect /dashboard
    Celery->>SMTP: Bienvenue email
    SMTP-->>Marie: E-mail reçu
```

5.2 Soumission demande d'audit (avec async)

Le client soumet le formulaire d'audit. Le front appelle `POST /api/audits/` avec le JWT en header. Django valide via le serializer DRF, persiste en BDD, enfile une tâche Celery d'envoi d'accusé de réception, puis répond `201 Created`. Le worker Celery dépile la tâche depuis Redis et envoie l'e-mail via SMTP.

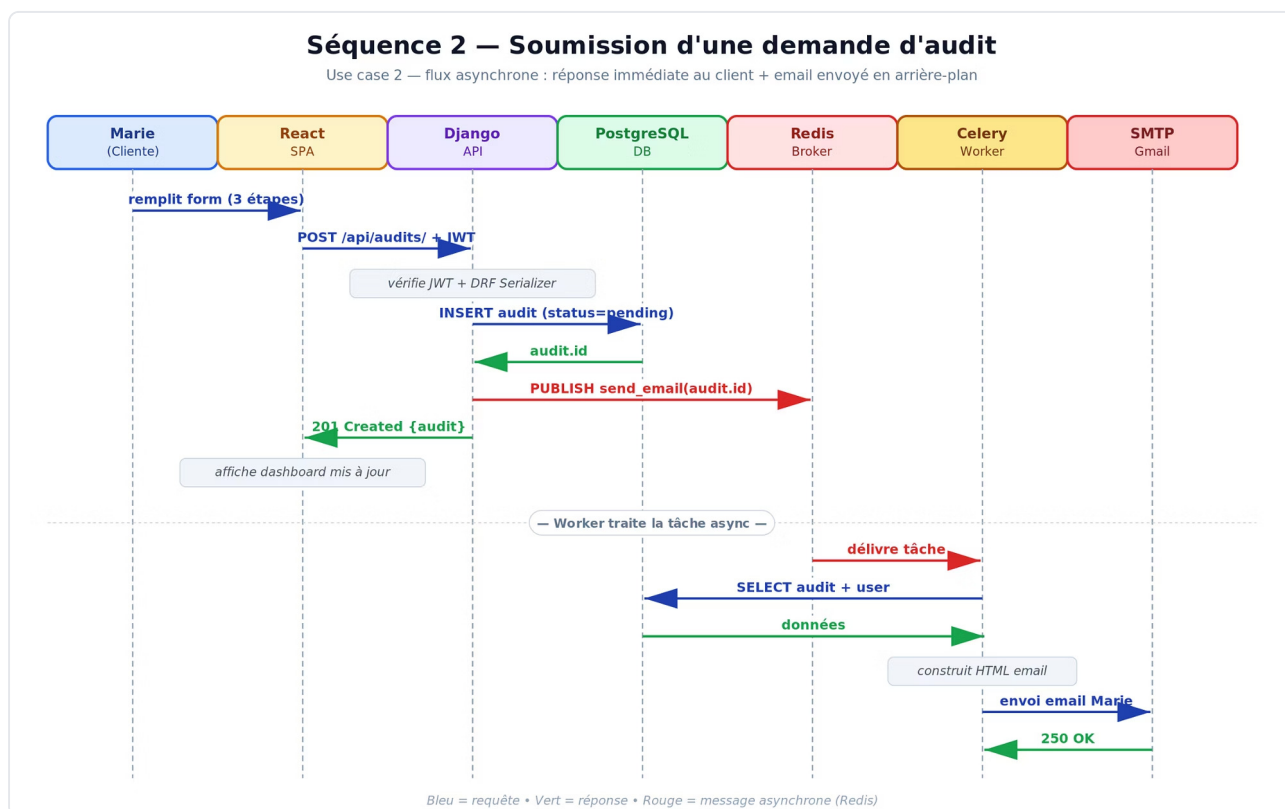


Figure 6 — Séquence Soumission demande (asynchrone Celery)

Messages échangés (en détail)

1. **Marie** → **AuditRequestForm** : sélectionne un pack et saisit `scope_notes` .
2. **AuditRequestForm** → **ApiClient** : `POST /api/audits/` , header `Authorization: Bearer <JWT>` , body `{pack_id, scope_notes}` .
3. **ApiClient** → **Django (AuditCreateView)** : middleware vérifie le JWT, DRF valide les champs.
4. **Django** → **PostgreSQL** : `INSERT INTO audits_auditrequest` avec `status='pending'` , `reference='CYB-2026-XXXX'` .
5. **Django** → **Redis (Celery broker)** : enfile `send_acknowledgment_email.delay(audit_id)` .
6. **Django** → **AuditRequestForm** : `201 Created` avec le détail de la demande.
7. **AuditRequestForm** → **ClientDashboard** : redirection avec toast « Demande envoyée ».
8. **Celery worker** ← **Redis** : dépile la tâche.
9. **Celery worker** → **SMTP Gmail** : envoie l'accusé de réception à Marie + notification à Karim.
10. **Celery worker** → **PostgreSQL** : `UPDATE notifications SET status='sent'` .

Mermaid (rendu sur GitHub)

```
sequenceDiagram
    actor Marie
    participant Front as AuditRequestForm
    participant API as Django /audits
    participant DB as PostgreSQL
    participant Redis
    participant Worker as Celery Worker
    participant SMTP

    Marie->>Front: Choisit pack + scope_notes
    Front->>API: POST /audits/ + JWT
    API->>API: Vérifier JWT + valider
    API->>DB: INSERT audit_request (status=pending)
    API->>Redis: enqueue send_ack_email(audit_id)
    API-->>Front: 201 Created {audit}
    Front-->>Marie: Redirect dashboard + toast
    Worker->>Redis: dequeue task
    Worker->>SMTP: Email accusé de réception
    Worker->>DB: UPDATE notification status=sent
    SMTP-->>Marie: Email reçu
```

5.3 Génération du rapport PDF (admin)

L'admin clique sur « Generate PDF report ». Le front appelle `POST /api/audits/{id}/generate-report/` . Django crée l'AuditReport et enfile la tâche Celery, retournant `202 Accepted` . Le worker Celery rend le template HTML, WeasyPrint produit le PDF, le fichier est stocké, et une notification e-mail est envoyée au client.

Séquence 3 — Génération du rapport PDF (admin)

Use case 3 — l'admin déclenche, le worker génère, le client est notifié et récupère

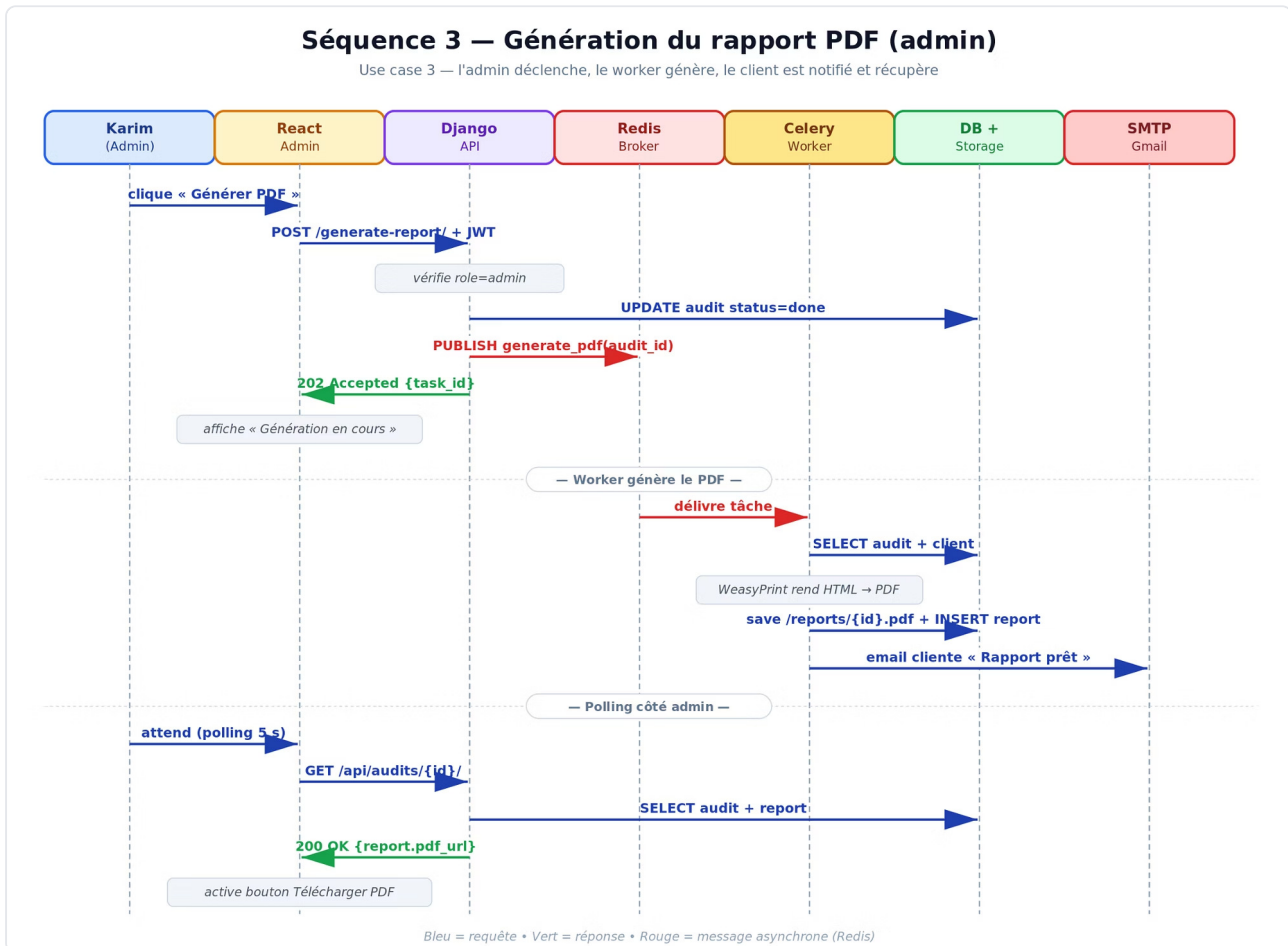


Figure 7 — Séquence Génération du rapport PDF

Messages échangés (en détail)

1. **Karim** → **AdminRequestDetail** : clique « Generate PDF report » avec les findings saisis.
2. **AdminRequestDetail** → **ApiClient** : `POST /api/audits/{id}/generate-report/`, body `{findings, summary, security_score}`.
3. **ApiClient** → **Django (PdfReportService)** : vérifie permission `IsAdmin`, vérifie `status='done'`.
4. **Django** → **PostgreSQL** : `INSERT INTO reports_auditreport` avec `pdf_path=NULL` (encore en cours).
5. **Django** → **Redis** : enfile `generate_pdf_task.delay(report_id)`.
6. **Django** → **AdminRequestDetail** : `202 Accepted` avec `{report_id, status: "generating"}`.
7. **Celery worker** ← **Redis** : dépile la tâche.
8. **Celery worker** → **Django templates** : rend `report.html` avec le contexte (findings, score, recommandations).
9. **Celery worker** → **WeasyPrint** : convertit HTML+CSS en PDF binaire.
10. **Celery worker** → **Storage** : sauvegarde dans `/media/reports/CYB-2026-0042.pdf`.
11. **Celery worker** → **PostgreSQL** : `UPDATE auditreport SET pdf_path=...`.
12. **Celery worker** → **SMTP Gmail** : envoie e-mail « Rapport disponible » à Marie.
13. **Marie** → **ClientDashboard** : voit le lien « Download report » apparaître (polling 30 s).
14. **Marie** → **Django** : `GET /api/audits/{id}/report/` avec JWT.
15. **Django** → **Marie** : URL signée 15 min, streaming du PDF.

Mermaid (rendu sur GitHub)

```
sequenceDiagram
    actor Karim
    participant Front as AdminRequestDetail
    participant API as Django /audits
    participant DB as PostgreSQL
    participant Redis
    participant Worker as Celery Worker
    participant WP as WeasyPrint
    participant Storage
    participant SMTP
    actor Marie

    Karim->>Front: Clic Generate PDF report
    Front->>API: POST /audits/{id}/generate-report/
    API->>DB: INSERT auditreport (pdf_path=NULL)
    API->>Redis: enqueue generate_pdf_task
    API-->>Front: 202 Accepted
    Worker->>Redis: dequeue
    Worker->>Worker: render report.html
    Worker->>WP: HTML+CSS to PDF
    WP-->>Worker: PDF bytes
    Worker->>Storage: save /media/reports/CYB-...pdf
    Worker->>DB: UPDATE pdf_path
    Worker->>SMTP: Email Rapport disponible
    SMTP-->>Marie: Email reçu
    Marie->>API: GET /audits/{id}/report/ + JWT
    API-->>Marie: Stream PDF (signed URL 15min)
```

6. Documentation des APIs

6.1 APIs externes consommées

API externe	Usage	Justification
SMTP Gmail	Envoi des emails transactionnels (confirmation, changement statut, livraison rapport)	Gratuit jusqu'à 500 emails/jour, configuration simple, fiable. Migration possible vers SendGrid/Mailgun en V2.
Let's Encrypt	Certificats HTTPS automatiques	Gratuit, intégré nativement dans Vercel et Railway.

6.2 API interne — Endpoints REST

Convention : préfixe `/api/`, JSON pour requêtes et réponses, JWT dans `Authorization: Bearer <token>`, codes HTTP standards.

6.2.1 Authentification — `/api/auth/`

Méthode	URL	Auth	Description	Body	Réponse 2xx	Erreurs
POST	<code>/api/auth/register/</code>	—	Crée un compte client	{email, password, company_name, company_size, sector}	201 {user, access, refresh}	400, 409 email exists
POST	<code>/api/auth/login/</code>	—	Connexion + tokens JWT	{email, password}	200 {access, refresh, user}	401, 429 rate limit
POST	<code>/api/auth/refresh/</code>	—	Renouvelle access	{refresh}	200 {access}	401 refresh expired
POST	<code>/api/auth/logout/</code>	✓	Invalide le refresh	{refresh}	204	401
POST	<code>/api/auth/password-reset/</code>	—	Envoie email de reset	{email}	200 (toujours, anti-leak)	—

6.2.2 Demandes d'audit — `/api/audits/`

Méthode	URL	Auth	Rôle	Description
GET	<code>/api/audits/</code>	✓	client / admin	Liste paginée (client → ses demandes / admin → toutes)
POST	<code>/api/audits/</code>	✓	client	Crée une nouvelle demande

Méthode	URL	Auth	Rôle	Description
GET	/api/audits/{id}/	✓	owner / admin	Détail d'une demande
PATCH	/api/audits/{id}/	✓	admin	Modifie statut ou notes
DELETE	/api/audits/{id}/	✓	admin	Archive (soft delete)
POST	/api/audits/{id}/generate-report/	✓	admin	Lance la génération PDF (Celery)
GET	/api/audits/{id}/report/	✓	owner / admin	Télécharge le PDF (binaire)

Exemple — Création d'une demande

```
POST /api/audits/
Authorization: Bearer eyJhbGc...
Content-Type: application/json

{ "pack_id": 2, "scope_notes": "Audit du serveur de fichiers + 12 postes Windows." }

HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "8f2a...c9",
  "client": "user-uuid",
  "pack": {"id": 2, "name": "standard"},
  "status": "pending",
  "scope_notes": "Audit du serveur ...",
  "created_at": "2026-05-18T09:24:11Z"
}
```

6.2.3 Packs d'audit — /api/packs/

Méthode	URL	Auth	Description
GET	/api/packs/	—	Liste publique des 4 packs
GET	/api/packs/{id}/	—	Détail d'un pack

6.2.4 Formations — /api/training/

Méthode	URL	Auth	Description
GET	/api/training/modules/	✓	Liste des modules disponibles
GET	/api/training/modules/{id}/	✓	Contenu d'un module
POST	/api/training/modules/{id}/start/	✓	Marque comme commencé
POST	/api/training/modules/{id}/complete/	✓	Marque comme terminé

6.2.5 Profil utilisateur — `/api/users/`

Méthode	URL	Auth	Description
GET	<code>/api/users/me/</code>	✓	Profil de l'utilisateur courant
PATCH	<code>/api/users/me/</code>	✓	Modifie son propre profil
POST	<code>/api/users/me/change-password/</code>	✓	Change son mot de passe
DELETE	<code>/api/users/me/</code>	✓	Supprime son compte (RGPD)

6.3 Conventions transversales

- **Pagination** : `?page=1&page_size=20`, réponse `{count, next, previous, results}`
- **Filtres** : `?status=pending&ordering=-created_at`
- **Erreurs** : format unifié `{error: "code", message: "Message FR", details: {...}}`
- **Rate limiting** : 5 req/min sur `/auth/login/` et `/auth/password-reset/` (IP-based)
- **Versioning** : pas de v1 dans l'URL au MVP ; en V2 si breaking changes → `/api/v2/`

7. Stratégie SCM & QA

7.1 Source Code Management (Git)

Hébergeur : GitHub — repo `Tommy-JOUHANS/cyberaudit-solutions` (privé jusqu'à la soutenance, puis public).

Stratégie de branches (Git Flow allégé)

```
main          ← production uniquement (protégée, déploiement continu Vercel/Railway)
develop       ← branche d'intégration (CI obligatoire)
feature/<nom> ← nouvelles fonctionnalités (1 PR = 1 feature)
fix/<nom>      ← corrections de bugs
hotfix/<nom>   ← correctifs urgents directement sur main
release/<v>    ← stabilisation avant release (à partir de W11)
```

Convention de commits — Conventional Commits

```
feat(audits): ajout du formulaire de demande
fix(auth): corrige le refresh token JWT expiré
docs(readme): met à jour la stack technique
test(audits): ajoute tests unitaires CRUD
chore(deps): bump Django 5.0.2 → 5.0.3
refactor(reports): extrait PDFGenerator en service
```

Règles de Pull Request

Règle	Détail
Revue obligatoire	Chaque PR validée par l'autre membre avant merge
CI verte	Tests + lint OK avant merge
Pas de push direct	<code>main</code> et <code>develop</code> protégées par GitHub Branch Protection
Description structurée	« Ce qui change / Pourquoi / Comment tester »
Squash merge	1 PR = 1 commit dans <code>develop</code> (historique propre)

7.2 Stratégie QA

Pyramide de tests

```
      /\
     /E2E\      5 %  - Cypress (parcours critiques)
    /-----\
   / Intégr. \ 20 %  - Tests intégration API + DB (Django + DRF)
  /-----\
 / Unitaires  \ 75 %  - Jest (front) + pytest (back)
/-----\
```

Outils retenus

Couche	Outil	Cible
Tests unitaires back	pytest + pytest-django	Modèles, services, serializers
Tests API back	pytest + APIClient (DRF)	Endpoints auth, audits, reports
Tests unitaires front	Jest + React Testing Library	Composants, hooks, utils
Tests E2E	Cypress	Inscription → demande → email (mock)
Tests sécurité	Postman (collection) + bandit	XSS, SQLi, brute-force, JWT tampering
Tests UX	3 sessions utilisateurs (W7-W10)	Persona Marie, parcours < 3 min
Coverage	coverage.py + jest --coverage	Cible : 80 % min apps critiques
Lint	ruff + black ; eslint + prettier	Pre-commit hook
Type checking	mypy ; propTypes ou TS (V2)	À partir de W4

7.3 Tests sécurité manuels (W7-W10)

Test	Méthode	Résultat attendu
XSS persistant	<code><script>alert(1)</script></code> dans scope_notes	Échappé / sanitized
SQL injection	<code>' OR 1=1 --</code> dans email login	Aucune fuite, ORM bind
Brute-force	100 tentatives login	Bloqué dès la 6 ^e (rate-limit)
JWT tampering	Modifier <code>role</code> dans le payload	Signature invalide → 401
CSRF	POST sans header Authorization	401
HSTS / CSP	Headers HTTP en prod	Présents et stricts

7.4 Pipeline CI/CD

GitHub Actions

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup-python 3.11
      - install: pip install -r backend/requirements-dev.txt
      - lint:    ruff check . && black --check .
      - test:    pytest --cov=apps --cov-fail-under=80
```

```

frontend:
  runs-on: ubuntu-latest
  steps:
    - checkout
    - setup-node 18
    - install: npm ci
    - lint: npm run lint
    - test: npm test -- --coverage
    - build: npm run build

```

Déploiement continu

Environnement	Déclencheur	Cible	URL
Preview	Toute PR	Vercel preview + Railway branch deploy	pr- XX.preview.cyberaudit.app
Staging	Merge sur develop	Vercel + Railway staging	staging.cyberaudit.app
Production	Merge sur main (release tag)	Vercel + Railway prod	cyberaudit.app

Stratégie de release

- **Versioning SemVer** : 0.1.0 (W4 MVP partiel) → 0.5.0 (W6 async) → 1.0.0 (W12 soutenance)
- **Tag git**